315 PLOTS API Reference

Oct 13, 2026 · @Reltime Lab

Overview

The 315 PLOTS API is one REST/JSON service that the Kotlin app calls for login, plot booking, payments, referral earnings and admin work. Every screen in the app maps to one or two calls listed here.

Item	Value
Local base URL	`http://localhost:3000`
Production base URL	`https://<your-api-host>` (set at deploy)
Interactive docs (Swagger UI)	`<base>/docs`
OpenAPI spec (JSON)	`<base>/docs-json`, also committed as `backend/openapi.json`
Health check	`GET /health` returns `{"status":"ok"}`
Format	JSON in and out; file uploads use `multipart/form-data`

Two roles. `USER` is every customer: they browse and book plots. A USER becomes a partner when they apply and an admin approves them; partners also share a referral code, register leads and earn commission. `ADMIN` runs operations through the `/admin/**` endpoints. A USER calling an admin endpoint gets `403`.

Server is the source of truth. Plot availability, booking status, payment status, commission and payout amounts all come from the API. The app never calculates money or decides a payment succeeded.

Conventions

Every endpoint follows the same six rules, so the app can share one network layer.

1. **Auth header.** Send `Authorization: Bearer <accessToken>` on every call except `/auth/*`, `/health` and `/webhooks/*`. Access tokens last 15 minutes.
2. **Money is integer paise.** Every amount field ends in `Paise`: `amountPaise: 2100000` means ₹21,000. Show rupees as paise ÷ 100. Never send or store floats.

3. **Pagination.** List endpoints take `?page=0&size=20` (size 1 to 50) and return `{ "items": [...], "page": 0, "size": 20, "total": 134 }`.
4. **Errors.** Every error has the same shape. Branch on `code`, show `message`, and log `requestId`:

   ```
   { "code": "PLOT_UNAVAILABLE", "message": "This plot is no longer available", "requestId": "3f0c…" }
   ```

5. **Request IDs.** Every response carries an `X-Request-Id` header. You can send your own and the server echoes it, which links app logs to server logs.
6. **Dates** are ISO-8601 UTC strings, for example `2026-10-13T09:30:00.000Z`. Field names are camelCase.

HTTP status codes

Status	Meaning	What the app should do
200 / 201	Success (POST that creates returns 201)	Use the body
400	Validation failed or a business rule refused the request	Show `message`; fix input
401	Missing or expired token	Call `POST /auth/refresh` once, retry; if that fails, go to login
403	Wrong role, or partner not approved, or account disabled	Show `message`
404	Not found, or it belongs to another user	Show not found
409	State changed (plot taken, already exists)	Refresh the screen
429	Rate limited	Wait and retry; for OTP show the resend timer
501	Feature not configured (Google login, eSign)	Hide the feature
502	Payment or payout provider failed	Show retry
503	Bookings paused by admin	Show a banner

Error codes the app should handle

Code	When
`VALIDATION_FAILED`	A field is missing or malformed; `message` lists the fields
`TOKEN_INVALID`	Access or refresh token expired
`TOKEN_REUSED`	An old refresh token was used again; every session is revoked, send the user to login
`ACCOUNT_DISABLED`	Admin deactivated the account
`OTP_EXPIRED`, `OTP_INVALID`, `OTP_LOCKED`, `OTP_RATE_LIMITED`	OTP problems; see Authentication
`INVALID_REFERRAL`	Referral code unknown or partner not approved
`PROFILE_INCOMPLETE`	Name missing; send the user to the profile screen
`PARTNER_NOT_APPROVED`	Partner-only endpoint called by a non-approved user
`PLOT_UNAVAILABLE`	Plot was taken; refresh the list
`PENDING_BOOKING_EXISTS`	User already has an unpaid checkout
`HOLD_EXPIRED`	The 15-minute hold ran out; start a new booking
`BOOKINGS_PAUSED`	Admin paused bookings
`SIGNATURE_INVALID`	Razorpay signature did not match
`PROVIDER_ERROR`	Razorpay or Cashfree failed; retry
`CONFLICT`	Someone else changed the same record; refresh
`RATE_LIMITED`	Too many requests from this IP

Rate limits

Each IP may make 120 requests per minute. OTP requests are capped at 30 per minute per IP and 3 per 15 minutes per mobile number; each OTP allows 5 attempts.

Authentication

Users sign in with mobile + OTP or Google; both return the same token pair, and new users are created automatically on first sign-in.

Method	Path	Auth	Purpose
POST	`/auth/otp/request`	none	Send a 6-digit OTP by SMS
POST	`/auth/otp/verify`	none	Check the OTP, create the user if new, return tokens
POST	`/auth/google`	none	Sign in with a Google ID token
POST	`/auth/refresh`	none	Swap a refresh token for a new pair
POST	`/auth/logout`	none	Revoke this device's refresh token
GET	`/me/bootstrap`	Bearer	Splash-screen routing data

Request an OTP

`POST /auth/otp/request`

```
{ "mobile": "9876543210" }
```

Response `200`:

```
{ "expiresInSec": 300, "resendAfterSec": 30 }
```

The mobile must be a 10-digit Indian number starting with 6–9, without +91. A fourth request within 15 minutes returns `429 OTP_RATE_LIMITED`.

Verify the OTP

`POST /auth/otp/verify`

```
{
  "mobile": "9876543210",
  "code": "482913",
  "referralCode": "P7KQ2XM",
  "leg": "L",
  "deviceId": "pixel-7-abc"
}
```

`referralCode`, `leg` and `deviceId` are optional. `referralCode` only applies to a brand-new user and makes that partner their sponsor. `leg` (`L` or `R`) picks the side of the sponsor's team; if omitted, the server places the user on the side with fewer direct members.

Response `200`:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs…",
  "refreshToken": "q8V2…",
  "expiresIn": 900,
  "isNewUser": true,
  "profileComplete": false
}
```

Errors: `OTP_INVALID` (wrong code), `OTP_EXPIRED` (older than 5 minutes or none requested), `OTP_LOCKED` (5 wrong attempts), `INVALID_REFERRAL`, `ACCOUNT_DISABLED`. When `profileComplete` is false, send the user to the profile screen before booking.

Google sign-in

`POST /auth/google` with `{ "idToken": "<Google ID token>", "referralCode"?, "leg"?, "deviceId"? }`. The response matches OTP verify. The server checks the token against Google and requires a verified email. If Google client IDs are not configured, it returns `501 NOT_CONFIGURED`.

Refresh and logout

`POST /auth/refresh` with `{ "refreshToken": "…" }` returns a **new** access and refresh token; the old refresh token stops working. Refresh tokens last 30 days.

Important for the app: run refresh once at a time. If two requests refresh with the same token, the second looks like theft: the server returns `TOKEN_REUSED` and signs the user out on every device. Use a single mutex or OkHttp `Authenticator` that queues requests while one refresh runs.

`POST /auth/logout` with `{ "refreshToken": "…" }` returns `{ "ok": true }`.

Bootstrap (splash screen)

`GET /me/bootstrap` returns everything the splash screen needs to route:

```
{
  "user": { "id": "…", "role": "USER", "name": "Asha Patil", "partnerStatus":
"APPROVED", "referralCode": "P7KQ2XM", "active": true, "…": "…" },
  "profileComplete": true,
  "minAppVersion": "1.0.0",
  "bookingsEnabled": true,
  "support": { "phone": "", "email": "" }
}
```

Route: `401` → login; app older than `minAppVersion` → force-update screen; `profileComplete: false` → profile; `role: ADMIN` → admin panel; otherwise home. Show partner tabs only when `partnerStatus` is `APPROVED`.

Profile and partner

Profile endpoints work for every USER; partner endpoints (network, leads, earnings) return `403 PARTNER_NOT_APPROVED` until an admin approves the user.

Method	Path	Purpose
GET	`/me`	Profile plus nominee
PUT	`/me`	Update name (required), email, address
PUT	`/me/nominee`	Set nominee: name, relation, optional mobile
POST	`/me/devices`	Register the FCM push token
PUT	`/me/sponsor`	Add a referral code later (only before the first booking)
POST	`/partners/apply`	Apply to become a partner
GET	`/partners/me`	Partner status, referral code and share link
GET	`/partners/network?level=0`	Team members down to 3 levels
POST	`/leads`	Register a prospect's name and mobile
GET	`/leads`	Leads with their progress

Profile

`PUT /me`

```
{ "name": "Asha Patil", "email": "asha@example.com", "address": "Baner, Pune"
}
```

Returns the full profile. Booking is blocked with `PROFILE_INCOMPLETE` until `name` is set. An email already used by another account returns `409 CONFLICT`.

`PUT /me/nominee` takes `{ "name": "Rahul Patil", "relation": "Spouse", "mobile": "9876500000" }`.

`POST /me/devices` takes `{ "token": "<FCM token>", "platform": "android" }` (or `ios`). Call it after every login and whenever Firebase rotates the token.

`PUT /me/sponsor` takes `{ "referralCode": "P7KQ2XM", "leg": "R" }`. It returns `409 SPONSOR_ALREADY_SET` if a sponsor exists, or `409 SPONSOR_LOCKED` after the user has booked.

Becoming a partner

`POST /partners/apply` (no body) moves the user from `NONE` or `REJECTED` to `PENDING` and returns `{ "partnerStatus": "PENDING" }`. The admin then approves or rejects; the user gets a notification either way.

`GET /partners/me`:

```
{
  "partnerStatus": "APPROVED",
  "referralCode": "P7KQ2XM",
  "referralLink": "https://315plots.app/r/P7KQ2XM",
  "directLeft": 4,
  "directRight": 2
}
```

`referralCode` and `referralLink` are `null` until approved.

Network

`GET /partners/network?level=0&page=0&size=20` lists the team. `level=0` means all three levels; `1`, `2` or `3` filters to one. Mobiles are masked.

```
{
  "items": [
    { "id": "…", "name": "Vikram", "mobile": "******3210", "partnerStatus":
"APPROVED", "level": 1, "side": "L", "hasBooking": true, "createdAt": "…" }
  ],
  "page": 0, "size": 20, "total": 9
}
```

`side` tells you which of the partner's two legs the member sits in, used for 70:30 matching.

Leads

`POST /leads` takes `{ "name": "Neha", "mobile": "9822000000", "note": "Met at site visit" }` and returns `201`. When that mobile later signs up without a code, the partner automatically becomes the sponsor. A mobile that already has an account returns `409 ALREADY_REGISTERED`.

`GET /leads` returns each lead with `status`: `NEW` (not signed up), `REGISTERED` (signed up) or `BOOKED` (has a confirmed booking).

Plots and plans

There are 315 plots at ₹50 lakh each, sold through 5 booking plans; only plots with status `AVAILABLE` can be booked.

Method	Path	Purpose
GET	`/plots?status=&block=&facing=&page=&size=`	Plot list and map data, plus a count per status
GET	`/plots/{id}`	One plot
GET	`/tiers`	Active booking plans with prices

`GET /plots?status=AVAILABLE&size=50`:

```
{
  "items": [
    { "id": "…", "plotNo": "P-001", "block": "A", "facing": "East",
"areaSqft": 1200, "pricePaise": 500000000, "lat": 18.52, "lng": 73.85,
"status": "AVAILABLE" }
  ],
  "page": 0, "size": 50, "total": 312,
  "summary": [ { "status": "AVAILABLE", "count": 312 }, { "status": "BOOKED",
"count": 3 } ]
}
```

Use `lat` / `lng` for map pins. A plot whose 15-minute hold has lapsed already shows as `AVAILABLE`. Status values: `AVAILABLE`, `HELD` (someone is paying), `BOOKED`, `BLOCKED` (admin removed it from sale).

`GET /tiers` returns the plans in display order. Seeded values from the business plan:

code	name	amountPaise	Price	L1 / L2 / L3 commission
`EXPRESS`	Express Token	2100000	₹21,000	10% / 5% / 5%
`ADVANCE`	Advance Token	7100000	₹71,000	15% / 5% / 5%
`SILVER`	Silver	15100000	₹1,51,000	15% / 10% / 10%
`GOLD`	Gold	30000000	₹3,00,000	20% / 10% / 10%
`PLATINUM`	Platinum	65100000	₹6,51,000	25% / 15% / 10%

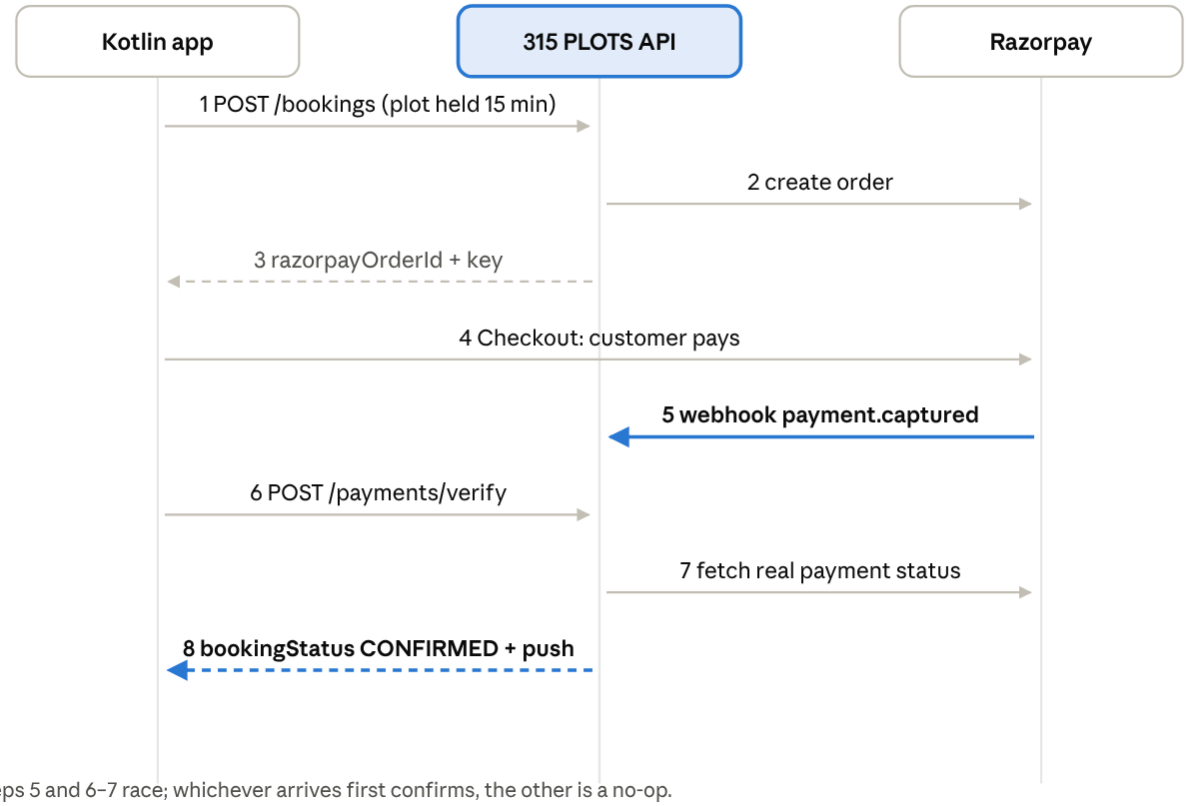
Commission rates come back as basis points (`l1Bps: 2000` = 20%). Admins can change prices; existing bookings keep the price they were booked at. Always show the price from this endpoint, never a hard-coded one.

Booking and payment

A booking holds the plot for 15 minutes while the customer pays through Razorpay; it becomes `CONFIRMED` only after the server verifies the payment with Razorpay.

Method	Path	Purpose
POST	`/bookings`	Hold a plot and open a Razorpay order
POST	`/payments/verify`	Hand the Razorpay result to the server after checkout
GET	`/bookings/{id}`	Booking with its payments and documents (poll this)
GET	`/bookings`	The user's bookings
POST	`/bookings/{id}/retry-payment`	New Razorpay order while the hold is still live
GET	`/payments`	Payment history
GET	`/payments/{id}`	One payment with its booking status
GET	`/payments/{id}/receipt`	Receipt data for a verified payment

A booking is confirmed only after the server verifies the payment



Steps 5 and 6–7 race; whichever arrives first confirms, the other is a no-op.

booking and payment sequence · app, API, Razorpay

The app talks to Razorpay only for checkout; the server learns the truth from Razorpay twice, by webhook and by direct fetch, and confirms on whichever comes first.

Step by step in the app

1. **Create the booking** from the review screen. `POST /bookings`:

```
{ "plotId": "…", "tierId": "…", "termsAccepted": true }
```

Response `201`:

```
{
  "bookingId": "…", "bookingRef": "BK-2026-000123",
  "paymentId": "…", "paymentRef": "PAY-2026-000124",
  "amountPaise": 30000000, "currency": "INR",
  "razorpayOrderId": "order_N8x…", "razorpayKeyId": "rzp_live_…",
  "holdExpiresAt": "2026-10-13T09:45:00.000Z"
}
```

Show a countdown to `holdExpiresAt`. Errors: `PLOT_UNAVAILABLE`, `PENDING_BOOKING_EXISTS`, `PROFILE_INCOMPLETE`, `BOOKINGS_PAUSED`, `PROVIDER_ERROR` (retry with retry-payment).

2. **Open Razorpay Checkout** with `key = razorpayKeyId`, `order_id = razorpayOrderId`, `amount = amountPaise`, `currency = INR`. Do not create orders in the app.
3. **Send the result to the server.** On Razorpay's success callback, `POST /payments/verify`:

   ```
   { "razorpayOrderId": "order_N8x…", "razorpayPaymentId": "pay_N8y…",
   "razorpaySignature": "9e1c…" }
   ```

   The server checks the signature, then asks Razorpay directly for the payment's real status. Response:

   ```
   { "id": "…", "internalRef": "PAY-2026-000124", "status": "VERIFIED",
   "amountPaise": 30000000, "providerPaymentId": "pay_N8y…", "bookingId":
   "…", "bookingRef": "BK-2026-000123", "bookingStatus": "CONFIRMED" }
   ```

4. **If not confirmed yet, poll.** If `bookingStatus` is still `PENDING_PAYMENT` (bank still processing), poll `GET /bookings/{id}` every 3 seconds for up to 2 minutes. Razorpay's webhook confirms it on the server in the background, and a push notification arrives too.
5. **On failure**, Razorpay's failure callback means nothing was charged. While the hold lasts, call `POST /bookings/{id}/retry-payment` for a fresh order (same response shape as step 1). After the hold ends it returns `HOLD_EXPIRED`; start again from step 1.

Never mark a booking paid from the app. Only the server's `CONFIRMED` status counts. If the customer pays after the hold ended and someone else took the plot, the server refunds the payment automatically and sets the booking to `REFUND_PENDING`.

Booking detail

`GET /bookings/{id}` returns `id`, `internalRef`, `status`, `amountPaise`, `holdExpiresAt`, `confirmedAt`, `plot` (`plotNo`, `block`, `facing`, `areaSqft`, `lat`, `lng`), `tier` (`code`, `name`), `payments[]` (each with `internalRef`, `status`, `providerPaymentId`, `failureReason`, `verifiedAt`) and `documents[]`. Another user's booking returns `404`.

Receipt

`GET /payments/{id}/receipt` (verified payments only) returns `receiptNo`, `amountPaise`, `providerPaymentId`, `verifiedAt`, `bookingRef`, `plotNo`, `tierName`, `customerName`, `mobile` and `email`. The app renders and shares it; the server does not generate a PDF.

Statuses

Booking status	Meaning
`PENDING_PAYMENT`	Plot held, waiting for payment
`CONFIRMED`	Paid and verified; plot is booked
`EXPIRED`	Hold ran out before payment
`CANCELLED`	Cancelled by admin
`REFUND_PENDING`	Refund started
`REFUNDED`	Money returned

Payment status	Meaning
`CREATED` / `PENDING`	Order opened, not paid
`VERIFIED`	Razorpay confirmed the capture
`FAILED`	Declined, superseded by a retry, or amount mismatch (`failureReason`)
`REFUND_PENDING` / `REFUNDED`	Refund in progress / done

Earnings and payouts

When a referred customer's booking is confirmed, the three partners above them earn commission, and every amount is shown as gross, 20% admin, 5% TDS and net. All endpoints here are partner-only.

Method	Path	Purpose
GET	`/commissions/summary`	Totals per status
GET	`/commissions?status=&kind=`	Commission ledger (also `kind=MATCHING`)
GET	`/commissions/{id}`	One entry with booking details and status history
GET	`/matching/summary`	Left/right team volume and carry-forward
GET	`/matching/runs`	Daily matching results

Method	Path	Purpose
GET	`/payouts/beneficiary`	Saved bank details (masked)
PUT	`/payouts/beneficiary`	Save bank account, IFSC and PAN
GET	`/payouts`	Payout history and amount ready to pay

How commission is earned

A Gold booking (₹3,00,000) by a customer whose sponsor chain is C → B → A pays:

Partner	Level	Rate	Gross	Admin 20%	TDS 5%	Net
C (direct sponsor)	1	20%	₹60,000	₹12,000	₹3,000	₹45,000
B	2	10%	₹30,000	₹6,000	₹1,500	₹22,500
A	3	10%	₹30,000	₹6,000	₹1,500	₹22,500

A partner who is not approved at that moment gets nothing for that level, and the share is not passed up. If the booking is later cancelled, a negative reversal entry cancels the commission; if it was already paid, the reversal is deducted from the next payout.

Ledger entries

`GET /commissions?status=APPROVED&page=0&size=20`:

```
{
  "items": [
    {
      "id": "…", "kind": "COMMISSION", "level": 1,
      "grossPaise": 6000000, "adminPaise": 1200000, "tdsPaise": 300000,
"netPaise": 4500000,
      "status": "APPROVED", "statusAt": "…", "reversesId": null, "createdAt":
"…",
      "bookingRef": "BK-2026-000123", "plotNo": "P-001", "tierName": "Gold"
    }
  ],
  "page": 0, "size": 20, "total": 1
}
```

`GET /commissions/{id}` adds `buyerName` and `history[]` (each `status`, `note`, `createdAt`). `GET /commissions/summary` returns `byStatus[]` with `status`, `count`, `grossPaise`, `adminPaise`, `tdsPaise`, `netPaise`.

Entry statuses move `PENDING` → `ELIGIBLE` (after a 7-day cooling period) → `APPROVED` (by admin) → `PAID`. `ON_HOLD` and `REVERSED` can happen along the way.

70:30 matching

`GET /matching/summary`:

```
{
  "leftVolumePaise": 30000000, "rightVolumePaise": 7100000,
  "carryLeftPaise": 30000000, "carryRightPaise": 7100000,
  "powerLeg": "L",
  "byStatus": [ { "status": "PENDING", "count": 1, "netPaise": 375000, "…":
"…" } ]
}
```

Every confirmed booking anywhere below a partner adds to that partner's left or right volume. A daily run at 01:30 IST matches volume in a 70:30 ratio and pays a bonus of 50% of the matched volume, minus the same deductions. The bonus appears in the ledger as `kind: MATCHING`. The exact matching rule still needs business confirmation.

Bank details and payouts

`PUT /payouts/beneficiary`:

```
{ "holderName": "Asha Patil", "accountNumber": "123456789012", "ifsc":
"HDFC0001234", "pan": "ABCDE1234F" }
```

The server registers the account with Cashfree and stores account number and PAN encrypted. It returns masked values: `{ "holderName": "Asha Patil", "accountMasked": "****9012", "ifsc": "HDFC0001234", "panMasked": "******234F", "status": "VERIFIED" }`. PAN is required because TDS is deducted.

`GET /payouts` returns the payout list (`internalRef`, `amountPaise`, `status`, `failureReason`, `createdAt`) plus `availablePaise`, the approved net amount waiting for the next payout. Payout statuses: `REQUESTED` → `PROCESSING` → `PAID` or `FAILED`. A failed payout returns its entries to the available amount.

Documents and notifications

Documents are private files the admin uploads for a customer, opened through a link that expires in 5 minutes; notifications are an inbox plus a push message for every important event.

Method	Path	Purpose
GET	`/documents`	The user's documents
GET	`/documents/{id}/url`	A 5-minute download link
POST	`/documents/{id}/sign`	eSign (returns `501` until a provider is chosen)
GET	`/notifications`	Inbox with unread count
POST	`/notifications/{id}/read`	Mark one read
POST	`/notifications/read-all`	Mark all read

Documents

`GET /documents` returns `items[]` with `id`, `type` (`AGREEMENT`, `RECEIPT`, `ALLOTMENT`, `OTHER`), `title`, `status` (`PENDING`, `SENT`, `SIGNED`, `REJECTED`, `EXPIRED`), `bookingRef`, `createdAt`.

`GET /documents/{id}/url` returns `{ "url": "https://…signed…", "expiresInSec": 300 }`. Open the URL immediately in a viewer or download it; request a new one each time, never cache it.

Notifications

`GET /notifications`:

```
{
  "items": [
    { "id": "…", "category": "BOOKING", "title": "Booking confirmed", "body":
"Plot P-001 is booked. Ref BK-2026-000123", "deepLink": "booking/7c1e…",
"readAt": null, "createdAt": "…" }
  ],
  "page": 0, "size": 20, "total": 12, "unread": 3
}
```

Categories: `BOOKING`, `PAYMENT`, `DOCUMENT`, `REFERRAL`, `COMMISSION`, `PAYOUT`, `ACCOUNT`, `GENERAL`.

The FCM push carries the same data in its `data` payload: `category`, `deepLink`, `notificationId`. Map each `deepLink` to a screen:

deepLink	Open
`booking/{id}`	Booking detail

deepLink	Open
`payment/{id}`	Payment status or receipt
`document/{id}`	Document centre, that document
`commission`	Commission dashboard
`matching`	70:30 matching screen
`payout/{id}`	Payout history
`partner`	Partner status / onboarding
empty	Notification inbox

Admin API

Everything under `/admin` needs an `ADMIN` token, and every change an admin makes is written to the audit log with before and after values.

The first admin is created on server start from the `BOOTSTRAP_ADMIN_MOBILE` setting; that person signs in with OTP like anyone else. List endpoints accept `page` and `size` and the filters shown.

Area	Method	Path	Body / filters	Purpose
Dashboard	GET	`/admin/dashboard`		Plots by status, sales by plan, partner counts, ledger and payout totals
Plots	GET	`/admin/plots`	`status`, `block`, `facing`	All plots
Plots	POST	`/admin/plots`	`plotNo`, `block`, `facing`, `areaSqft`, `pricePaise`, `lat`, `lng`	Add a plot
Plots	PUT	`/admin/plots/{id}`	same + `status` (`AVAILABLE` or `BLOCKED`)	Edit or block a plot; held/booked plots cannot change status
Plots	POST	`/admin/plots/import`	multipart `file` (CSV)	Bulk upsert by plot number

Area	Method	Path	Body / filters	Purpose
Plans	GET	`/admin/tiers`		All plans, including inactive
Plans	POST / PUT	`/admin/tiers`, `/admin/tiers/{id}`	`code`, `name`, `amountPaise`, `l1Bps`, `l2Bps`, `l3Bps`, `benefits`, `active`, `sort`	Create or edit a plan
Users	GET	`/admin/users`	`search`, `role`, `partnerStatus`	Search by name, mobile, email or code
Users	GET	`/admin/users/{id}`		User, bookings, earnings, bank details
Users	POST	`/admin/users/{id}/partner`	`action`: `APPROVE`, `REJECT`, `SUSPEND`; `note`	Partner decision; approval creates the referral code
Users	POST	`/admin/users/{id}/active`	`active`: true/false	Deactivate signs the user out everywhere
Bookings	GET	`/admin/bookings`	`status`, `userId`	All bookings
Bookings	GET	`/admin/bookings/{id}`		Booking, payments, commission entries
Bookings	POST	`/admin/bookings/{id}/cancel`	`refund`: true/false, `reason`	Cancel, free the plot, reverse commission, optionally refund
Payments	GET	`/admin/payments`	`status`, `userId`	All payments
Ledger	GET	`/admin/commissions`	`status`, `kind`, `userId`	All ledger entries
Ledger	POST	`/admin/commissions/approve`	`ids[]`, `note`	`ELIGIBLE` → `APPROVED`
Ledger	POST	`/admin/commissions/hold`	`ids[]`, `note`	Put on hold
Ledger	POST	`/admin/commissions/release`	`ids[]`, `note`	`ON_HOLD` → `ELIGIBLE`
Payouts	GET	`/admin/payouts`	`status`, `userId`	All payouts

Area	Method	Path	Body / filters	Purpose
Payouts	POST	`/admin/payouts`	`userIds[]` (optional)	Pay every partner with approved net above the minimum
Documents	POST	`/admin/documents`	multipart: `userId`, `bookingId`?, `type`, `title`, `file` (PDF/JPEG/PNG, 10 MB)	Upload and notify the customer
Documents	PATCH	`/admin/documents/{id}/status`	`status`	Mark signed, rejected or expired
Notify	POST	`/admin/notifications/broadcast`	`audience`: `ALL` or `PARTNERS`, `title`, `body`	Message everyone
Settings	GET	`/admin/config`		All business settings
Settings	PUT	`/admin/config/{key}`	`value`	Change one setting (validated)
Audit	GET	`/admin/audit`	`entity`, `entityId`	Who changed what
Jobs	GET	`/admin/jobs`	`dead`	Background jobs
Jobs	POST	`/admin/jobs/{id}/retry`		Retry a failed job

Approve, hold and release return `{ "requested": 5, "changed": 4 }`. Entries in the wrong status are skipped, not errors.

Plot CSV format (header required; status is never changed by import):

```
plot_no,block,facing,area_sqft,price_paise,lat,lng
P-001,A,East,1200,500000000,18.52,73.85
```

Settings that `PUT /admin/config/{key}` accepts:

Key	Default	Allowed
`bookings_enabled`	`true`	`true` / `false` (pause switch)
`hold_minutes`	`15`	5–60

Key	Default	Allowed
`cooling_days`	`7`	0–90
`min_payout_paise`	`10000`	whole number
`admin_bps` / `tds_bps`	`2000` / `500`	0–10000
`matching_power_pct` / `matching_weak_pct`	`70` / `30`	1–99
`matching_bonus_bps`	`5000`	0–10000
`min_app_version`	`1.0.0`	`x.y.z`
`support_phone` / `support_email`	empty	any

Webhooks

Razorpay and Cashfree call these endpoints, never the app; each request is signature-checked, and a repeated event is accepted but has no effect.

Path	Called by	Signature check	Events used
`POST /webhooks/razorpay`	Razorpay	`X-Razorpay-Signature` = HMAC-SHA256 of the raw body with the webhook secret	`payment.captured`, `payment.failed`, `refund.processed`
`POST /webhooks/cashfree`	Cashfree Payouts	`x-webhook-signature` = base64 HMAC-SHA256 of timestamp + raw body	`TRANSFER_SUCCESS`, `TRANSFER_FAILED`, `TRANSFER_REJECTED`, `TRANSFER_REVERSED`

Both return `200 { "ok": true }` once the event is stored and applied, or `401` for a bad signature, which makes the provider retry. Set these URLs in each provider's dashboard as `https://<api-host>/webhooks/razorpay` and `https://<api-host>/webhooks/cashfree`. Turn on auto-capture in Razorpay. The Cashfree field names and signature scheme must be checked against Cashfree's current docs before go-live.